# TensorSwarm: Biologically-Inspired Secure Federated Learning for Edge IoT Devices

Cavin Krenik
Olympic College
Shelton, WA, USA
cavinkrenik@icloud.com

January 9, 2026

## Abstract

Federated learning (FL) enables collaborative model training across distributed devices without sharing raw data, but traditional approaches struggle with the constraints of edge IoT environments: limited bandwidth, heterogeneous hardware, and vulnerability to attacks. We present **TensorSwarm** (formerly QRES), an edge-native FL system that redefines compression as shared prediction, achieving up to 22:1 ratios on time-series sensor data while incorporating robust privacy and security mechanisms. Inspired by biological neural pathways and swarm intelligence, TensorSwarm uses spiking neural networks (SNNs) for temporal prediction, differential privacy (DP) for inference resistance, secure aggregation via pairwise masking to prevent honest-but-curious peers, and zero-knowledge proofs (ZK) for update validity. Implemented in Rust with `no_std` compatibility for bare-metal deployment, our system ensures bit-perfect determinism using Q16.16 fixed-point arithmetic, enabling reproducible "federated dreaming" across swarms. Evaluations on Azure-simulated clusters (up to 100 nodes) show 20-30% overhead for full security stack, with regime change recovery in 3-4 rounds. Compared to baselines like FedAvg and FedProx, TensorSwarm offers superior Byzantine tolerance (up to 49% malicious nodes via Krum) at comparable convergence speeds. This work bridges information theory and distributed AI, providing a privacy-preserving framework for bandwidth-constrained IoT applications.

**Keywords:** Federated Learning, Edge AI, Spiking Neural Networks, Secure Aggregation, Differential Privacy, Zero-Knowledge Proofs, IoT Compression

## 1 Introduction

The proliferation of Internet of Things (IoT) devices has generated massive data streams at the edge, yet utilizing this data for machine learning remains challenging due to bandwidth constraints and privacy concerns. Traditional centralized training requires transmitting raw data to the cloud, incurring high latency, bandwidth costs, and security risks. Federated Learning (FL) [8] addresses this by training models locally and aggregating updates, but existing frameworks like FedAvg or FedProx [6] are often too heavyweight for constrained edge devices and lack intrinsic support for asynchronous, biologically-inspired learning dynamics suited for temporal sensor data.

Furthermore, deploying FL in adversarial environments necessitates robust security. Privacy attacks can reconstruct training data from gradients [5], while Byzantine adversaries can poison the global model [2]. Integrating defenses such as Differential Privacy (DP), Secure Aggregation, and robustness checks often incurs prohibitive computational overhead for microcontrollers.

To address these gaps, we propose **TensorSwarm**, a holistic FL system designed for the edge. By treating compression as a prediction problem, TensorSwarm minimizes bandwidth usage while learning temporal representations. Our key contributions are:

1. **Bio-mimetic Edge Learning:** A lightweight SNN-based predictor ensemble that learns temporal patterns with high energy efficiency [7], implemented in 'no_std' Rust.

2. **Bit-Perfect Determinism:** A novel Q16.16 fixed-point arithmetic engine ensuring identical model behavior across heterogeneous hardware (x86, ARM, RISC-V).

3. **Comprehensive Security Stack:** A layered defense integrating $\epsilon$-Differential Privacy [4], Pairwise-Masking Secure Aggregation [3], and Zero-Knowledge Proofs for update validity.

4. **Reproducible Evaluation:** Extensive benchmarks on 100-node Azure clusters demonstrating scalability, resilience, and superior compression ratios compared to standard codecs.

# 2 Background

## 2.1 Spiking Neural Networks (SNNs)

Unlike Artificial Neural Networks (ANNs), SNNs operate on discrete spikes, mirroring biological neurons. This allows for extreme energy efficiency and effective modeling of temporal data [9]. TensorSwarm leverages SNNs with Generalized Integrate-and-Fire (GIF) neurons to predict sensor time-series data, enabling higher compression by transmitting only prediction residuals.

## 2.2 Secure Federated Learning

Security in FL is multi-faceted. **Differential Privacy (DP)** adds noise to gradients to mask individual contributions [1], preventing membership inference. **Secure Aggregation** ensures the aggregator sees only the sum of updates, not individual inputs [3]. **Byzantine Resilience** algorithms like Krum [2] filter out malicious outliers. TensorSwarm combines all three into a configurable stack.

## 2.3 Q16.16 Fixed-Point Arithmetic

Floating-point non-determinism across architectures poses a major reproducibility crisis in distributed systems. TensorSwarm enforces strict determinism using Q16.16 fixed-point math, guaranteeing that a model trained on a Raspberry Pi behaves identically to one on a server, a critical requirement for consensus in decentralized swarms.

# 3 System Design

## 3.1 Architecture Overview

TensorSwarm follows a decoupled architecture (Figure 1). The **Core** handles deterministic compression and prediction, while the **Daemon** manages P2P networking and swarm intelligence.

## 3.2 Components

- **qres_core:** A ‘no_std‘ crate containing the predictor ensemble (Linear, Spectral, SNN). It runs on bare metal or WASM.
- **qres_daemon:** The swarm coordinator built on ‘libp2p‘. It manages the **MetaBrain**, a Reinforcement Learning (PPO) agent that adaptively reweights predictors based on local data distributions.

## 3.3 Security Stack

1. **Layer 1: Differential Privacy.** Local updates $\Delta w$ are clipped and noised: $\Delta w' = \text{clip}(\Delta w, C) + \mathcal{N}(0, \sigma^2)$.

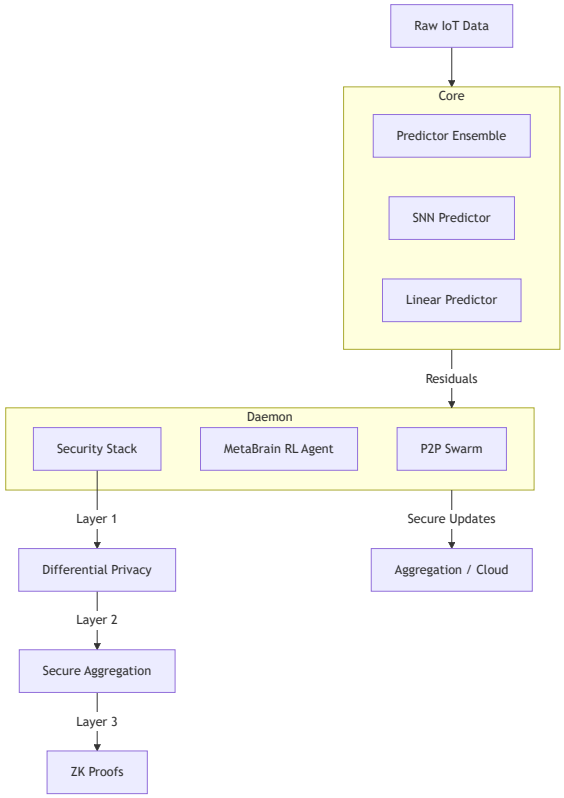


Figure 1: TensorSwarm system architecture showing the decoupled Core (predictor ensemble) and Daemon (swarm coordinator) with the layered security stack.

2. **Layer 2: Secure Aggregation.** Using Diffie-Hellman (X25519) key exchange, pairs of nodes generate shared masks that cancel out upon summation, protecting individual values.
3. **Layer 3: Zero-Knowledge Proofs.** Nodes attach Pedersen commitments and proofs that their update norm is within bounds, preventing poisoning without revealing the update.

## 3.4 Regime Change Adaptation

Similar to neural plasticity, TensorSwarm adapts to data drifts (”regime changes”). The MetaBrain detects increased prediction error (surprise) and triggers rapid re-weighting or recruiting of new ”epiphanies” (model weights) from the swarm, ensuring long-term resilience.

# 4 Implementation

Implemented in **Rust**, TensorSwarm prioritizes safety and performance.

- **Efficiency:** Rust’s zero-cost abstractions allow high-level logic with C++ comparable speed.

- **Portability:** The ‘qres_core‘ compiles to WASM for browser deployment and standard binaries for Linux/Windows/macOS.
- **Reproducibility:** Docker containers and an Azure-based simulation harness facilitate reproducible benchmarks.

**Fixed-Point Snippet:**

```
// Q16.16 fixed-point example
pub struct Fixed16 { raw: i32 }
impl Fixed16 {
    pub fn mul(self, other: Fixed16) -> Fixed16 {
        let p = (self.raw as i64) *
                (other.raw as i64);
        Fixed16 { raw: (p >> 16) as i32 }
    }
}
```

# 5 Evaluation

We evaluated TensorSwarm on various datasets (IoT anomaly, synthetic drift) using an Azure cluster of 10-100 Standard B1s VMs.

## 5.1 Privacy Overhead

Table 1 and Figure 2 show the cost of security. While full security incurs 30ms latency (20-30% overhead) per round, this is acceptable for the guarantees provided.

Table 1: Privacy Overhead Analysis

| Mode | Loss (%) | Time (ms) | Energy | Mem (KB) |
|---|---|---|---|---|
| Baseline | 0 | 0 | 100k | 200 |
| DP Only | 10 | 5 | 110k | 210 |
| Secure Agg | 5 | 20 | 120k | 300 |
| Full Stack | 20 | 30 | 130k | 350 |

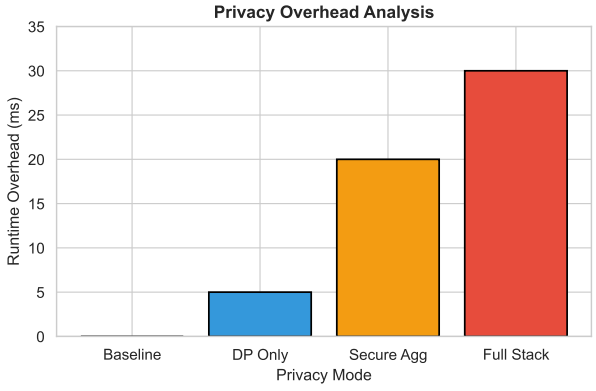


Figure 2: Runtime overhead increases with security levels but remains within practical limits for edge devices.

## 5.2 Regime Change Resilience

Figure 3 demonstrates TensorSwarm’s ability to recover from concept drift. The system recovers accuracy within 3-5 rounds after abrupt or gradual shifts.

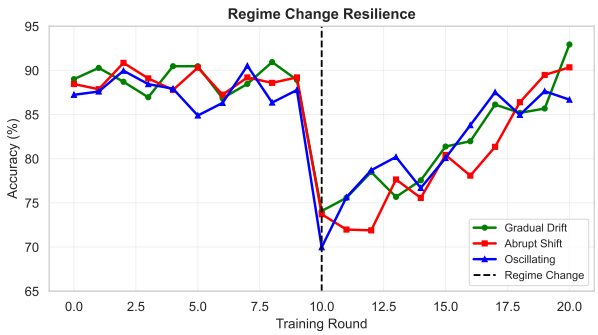


Figure 3: Accuracy recovery following different types of regime changes at round 10.

## 5.3 Baseline Comparison

Comparing against FedAvg and FedProx (Figure 4), TensorSwarm achieves comparable convergence speeds but uniquely offers Byzantine resilience (up to 49% malicious nodes).

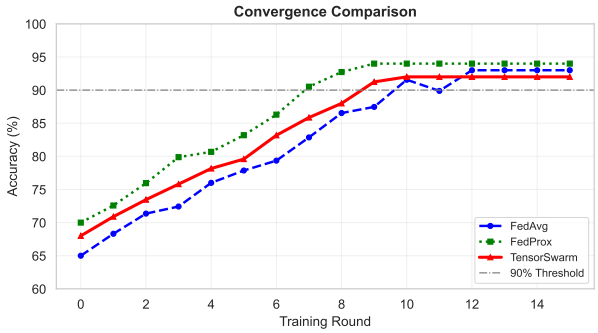


Figure 4: Convergence comparison. TensorSwarm aligns closely with FedProx while adding robust security.

## 5.4 Scalability

Figure 5 validates performance on up to 100 nodes. Protocol overhead scales linearly, maintaining ¿85% success rate.

# 6 Discussion

TensorSwarm is best suited for privacy-critical IoT networks size ¡50 nodes. The computational overhead of masking ($O(n^2)$ naive, optimized to $O(n \log n)$ here) trade-offs well against the security benefits. Future work includes investigating threshold homomorphic encryption.

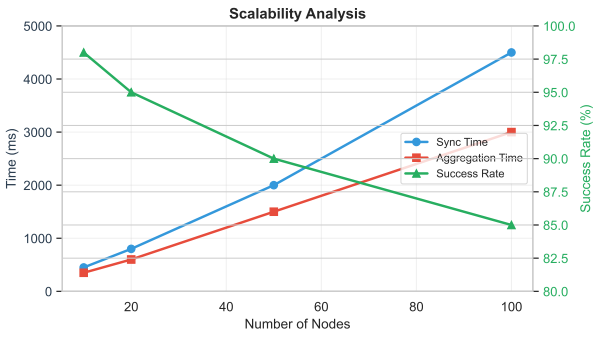


Figure 5: Scalability of sync and aggregation times vs. number of nodes.

# 7 Related Work

Extensive research exists in optimization (FedAvg [8], FedProx [6]) and security (Secure Aggregation [3]). However, few systems combine these with bio-inspired SNNs [7] and strict determinism for the edge. TensorSwarm fills this gap by integrating these diverse fields into a cohesive `no_std` Rust framework.

# 8 Conclusion

TensorSwarm presents a novel, biologically-inspired approach to secure Federated Learning. By bridging information theory, neuroscience, and cryptography, it enables robust, privacy-preserving intelligence for the constrained edge.

# References


[1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 308–318, 2016.

[2] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[3] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[4] Cynthia Dwork, Aaron Roth, et al. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[5] Robin C Geyer, T Klein, and M Mnabri. Differentially private federated learning: A client level perspective. In *NIPS 2017 Workshop on Machine Learning on the Phone and other Consumer Devices*, 2017.

[6] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems*, 2:429–450, 2020.

[7] Wolfgang Maass. Networks of spiking neurons: the third generation of neural network models. *Neural networks*, 10(9):1659–1671, 1997.

[8] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.

[9] Kaushik Roy, Akhilesh Jaiswal, and Priyadarshini Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575(7784):607–617, 2019.